Politecnico di Torino

Master's degree in
Computer Engineering

Accademic year: 2023-2024

Course notes

Machine learning and pattern recognition

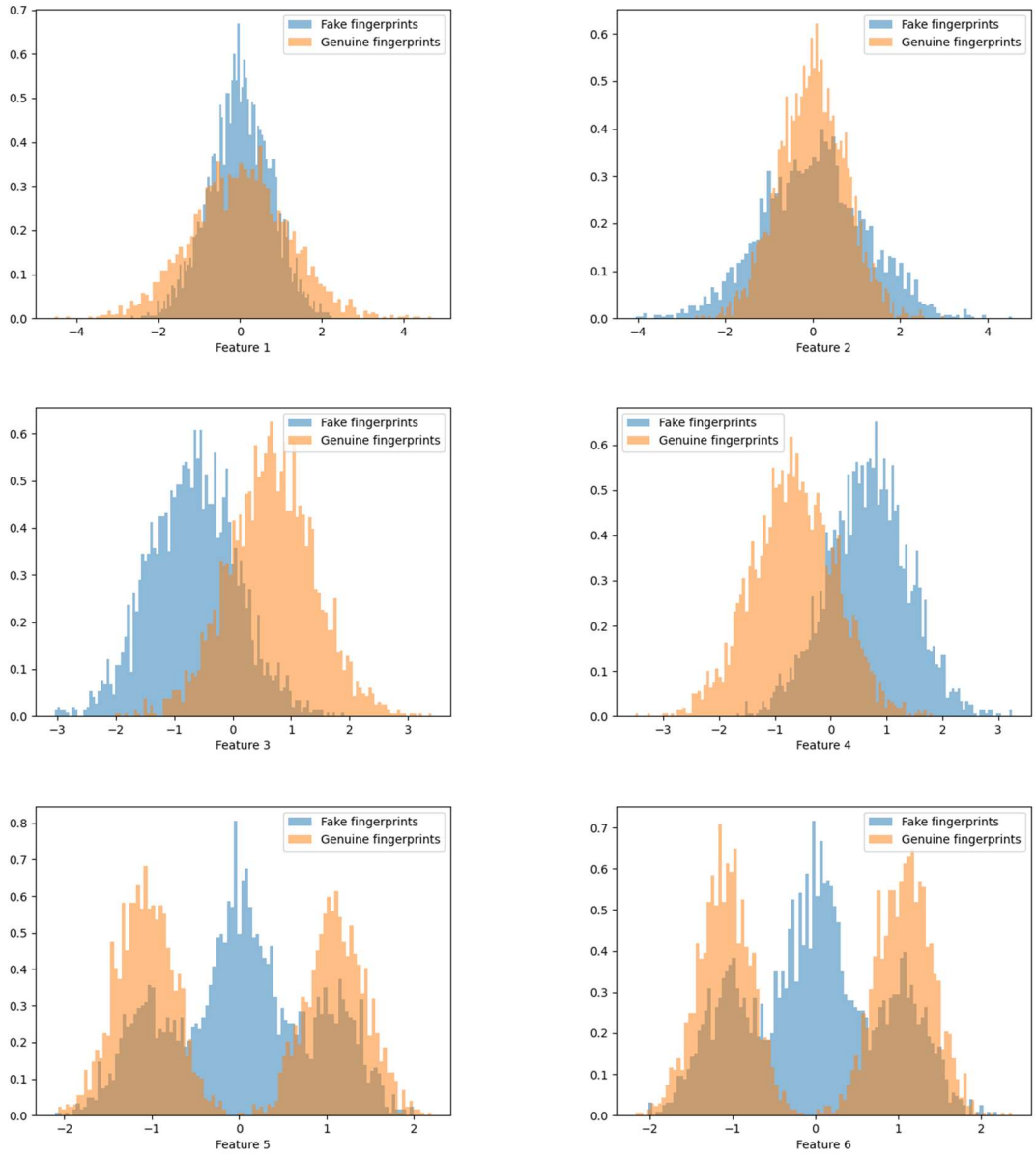
Professor:

Sandro Cumani

Author: Alberto Ciccomascolo
Student ID: 333968

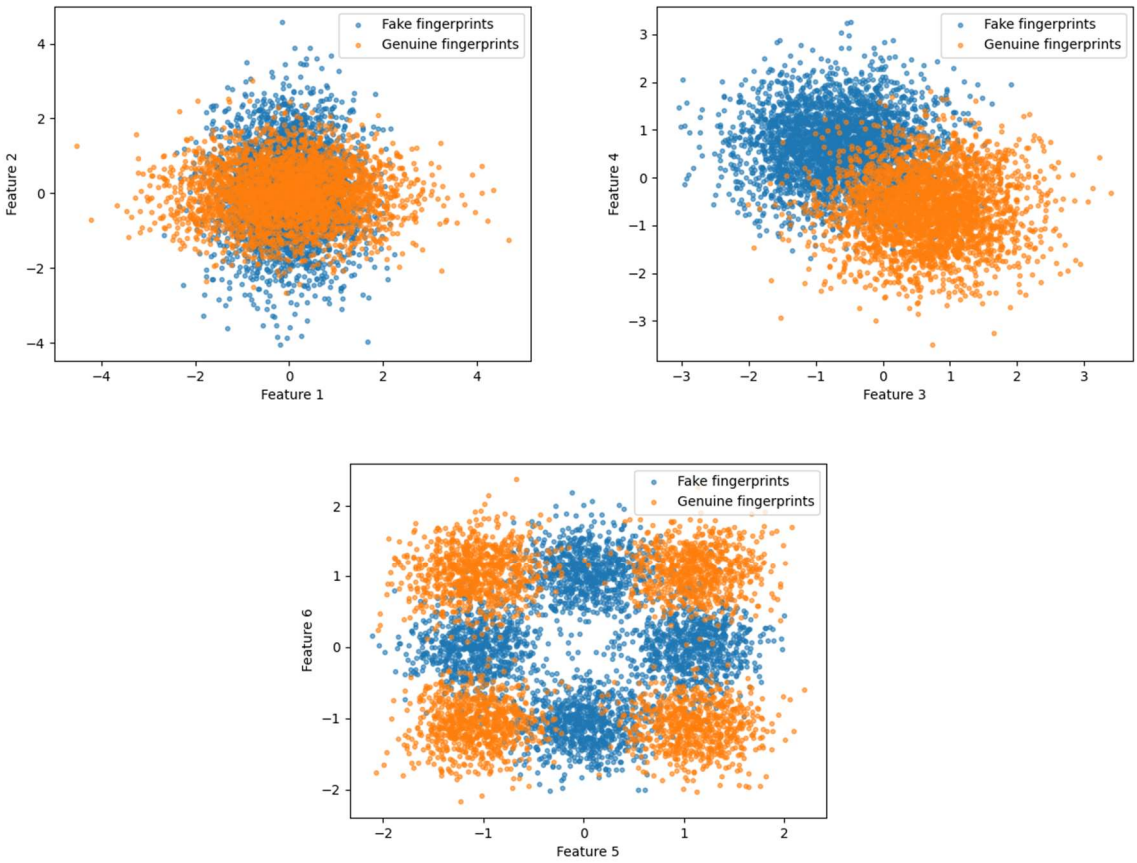
1. Dataset visualization

The training dataset is composed by 6000 samples, each one corresponding to a set of data related to a single fingerprint. Every sample is described by 6 different features and can belong to two classes, that is, a genuine (labelled as 1 or h_1) or fake (labelled as 0 or h_0) fingerprint.
The dataset contains a balanced presence of both classes (3010 genuine fingerprints and 2990 fake fingerprints).
The following histograms show the (normalized) distribution of all the samples with respect to each feature:



On a first look, the dataset seems to show similar distributions (sometimes mirrored) related to the samples among pair-wise features, that is, the feature pairs (f_1, f_2), (f_3, f_4), (f_5, f_6).
We can better observe the behavior of the pair-wise feature distributions by plotting the related scatter plots.

These pair-wise scatter plots are the following:



Moreover, in order to better understand the data, we can compute the values of the mean and the variance associated to each univariate feature distribution and divided for class (genuine or fake fingerprint).
The resulting values can be found in the following table:

	Mean		**Variance**	
	Genuine	*Fake*	*Genuine*	*Fake*
feature 1	5.44547838e-04	0.00287744	1.43023345	0.56958105
feature 2	-8.52437392e-03	0.01869316	0.57827792	1.42086571
feature 3	6.65237846e-01	-0.68094016	0.5489026	0.54997702
feature 4	-6.64195349e-01	0.6708362	0.55334275	0.53604266
feature 5	-4.17251858e-02	0.02795697	1.31776792	0.6800736
feature 6	2.39384879e-02	-0.0058274	1.28702609	0.70503844

By looking at the plots and at the previous table, we can clearly say that:

- The **first** and **second** features have a similar overall plot, but they show an opposite class distribution. In both cases the two classes overlap over an interval of values that is approximately equal to $[-2,2]$, where there is the higher sample frequency. This means that most of the feature values are covered by samples of both the class 0 (fake) and the class 1 (genuine). Only genuine class for feature f_1 and fake class for feature f_2 can assume values without any overlap with the opposite class; in fact, they have a higher spread.
  The scatter plot also denotes large samples overlapping, which does not involve the samples whose feature values are much larger and/or much lower than the mean.

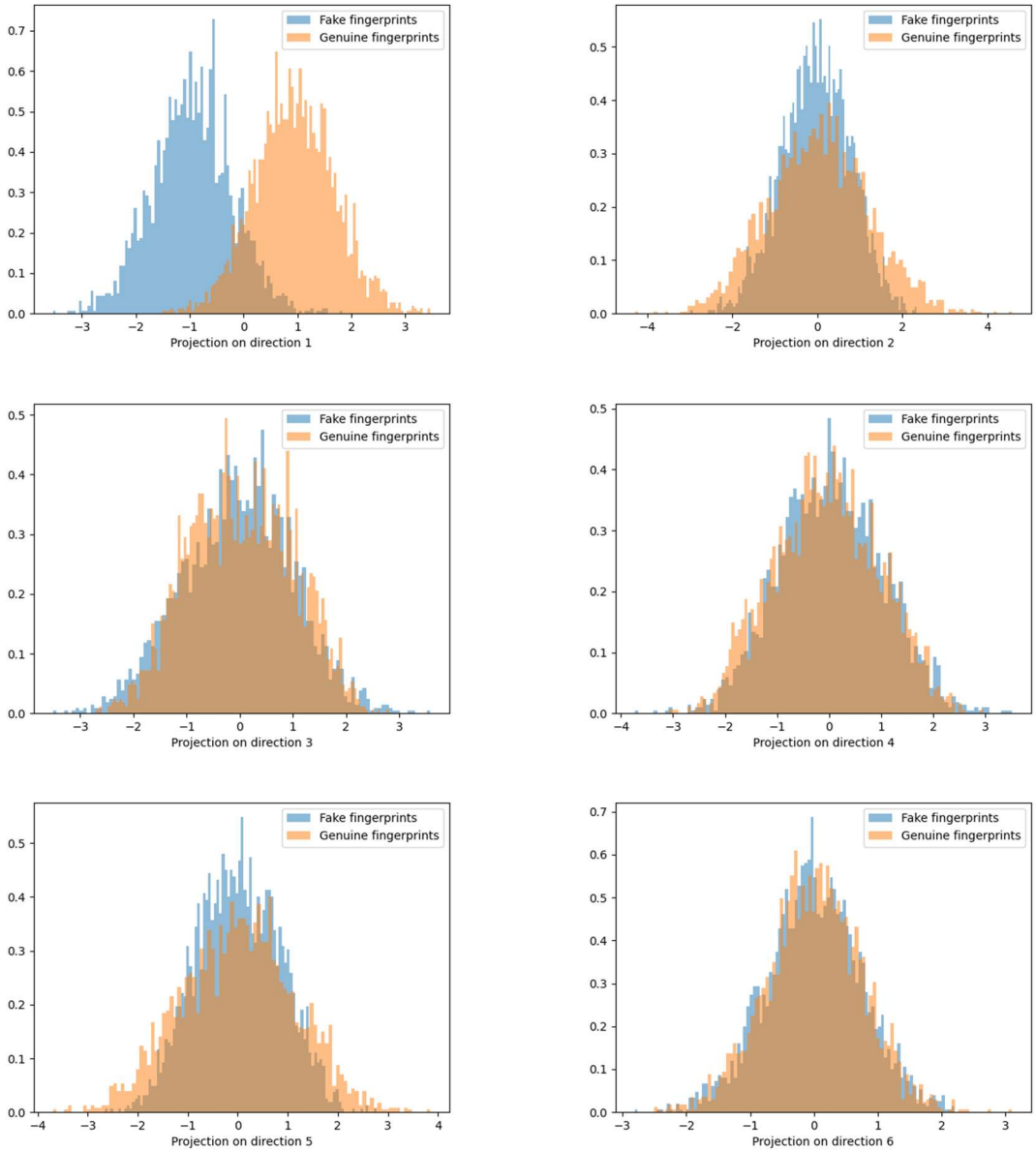
Despite this, both the classes for features f_1 and f_2 have a similar (but not exactly equal) mean, which tends to zero. Moreover, given that the two classes have a different peak height and given that the classes have an almost equal amount of samples, the variance of the two classes is obviously different. In fact, the code shows that for feature f_1 the variance of the genuine class is higher than the variance of the fake class and for feature f_2 the variance of the genuine class is lower than the variance of the fake class.
For both features, the two classes show essentially a single mode, that is located near the related mean value.

- The **third** and **fourth** features show again a similar graphical behavior as the overall plots have the same shape, but the class distributions are mirrored. In this case we can observe two peaks for each plot, one corresponding to a genuine sample and the other corresponding to a fake sample.
  The class overlap interval is the same for both features and corresponds approximately to the interval $[-2,2]$, but with a different class ratio (only near zero the two classes have a nearly equal frequency). Apart from this, the scatter plot indicates an overlapping that is smaller than the one of the previous case, denoting more uncorrelation.
  In this case the classes for features f_3 and f_4 show opposite mean values, which is a consequence of the mirrored distributions. The previous table also shows that the variance is similar both among different classes of the same feature and among different features of the same class.
- The **fifth** and **sixth** features have again the same overall plot shape, but this time the single classes also show an equal behavior among the two features, that is, the distributions are not mirrored. In fact, in both plots, we can observe a central peak corresponding to a fake sample and two main side peaks corresponding to genuine samples, which are symmetrical with respect to the origin.
  For both features the two classes spread along the same interval of values (which is again $[-2,2]$), meaning that for features f_5 and f_6 the there is (almost) no value of one class that cannot be also assumed by the other.
  As the scatter plot shows, the fact that the peak of a class corresponds to low frequency intervals of the other class, makes the overlapping area less impactful.
  Moreover, in both plots and for both classes, the related mean value is slightly different but always near to zero. On the other hand, features f_5 and f_6 have an higher variance for genuine class (because of the two side peaks) and a lower variance for fake class.

2. Dimensionality reduction

2.1 Principal Component Analysis (PCA)

The first approach for reducing the dimensionality of the dataset is applying PCA. We start by retrieving the 6 PCA directions u_i and then we project the dataset over each one of these directions (starting from the principal) by calculating $\widehat{D} = u_i^T D$, where D is the dataset and $\widehat{D}$ is the resulting projected dataset.
This means reducing the dimensionality of the problem from the initial value of $n = 6$ to $m = 1$.
The plots resulting from the projection of the dataset over the PCA directions (represented in descending order of variance retaining) are the following:

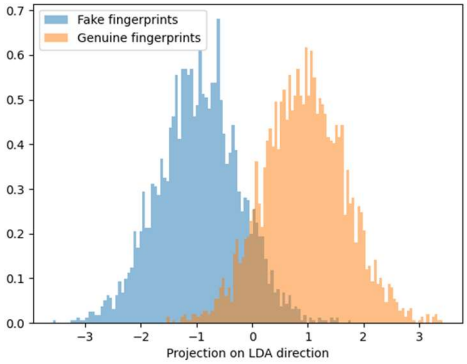


What we can notice from the previous plots is the fact that the less is the variance preserved by the projection direction, the less distinguishable are the classes distributions. In particular, the dataset

projected over the principal direction (direction 1 in the plot) is the one that is better able to separate the two classes into two clusters which have a slight overlapping but that still have two distinct peaks, from which we can derive the fact that genuine fingerprints are more associated with large (positive) values of the single projected feature while fake fingerprints are more associated with low (negative) values of the feature.
By projecting on the other directions we can clearly see that the two classes distributions become more and more undistinguishable and completely overlapped, making a classification impossible, since there are no relevant clusters.

2.2 Linear Discriminant Analysis (LDA)

A better way to reduce the dataset dimensionality preserving class-discriminant information is applying LDA, which prioritize the projection directions that allow to better separate the classes.
In this case, since we are dealing with a binary problem, LDA can retrieve only one discriminant direction over which to project the dataset.
After the projection we obtain the following plot:

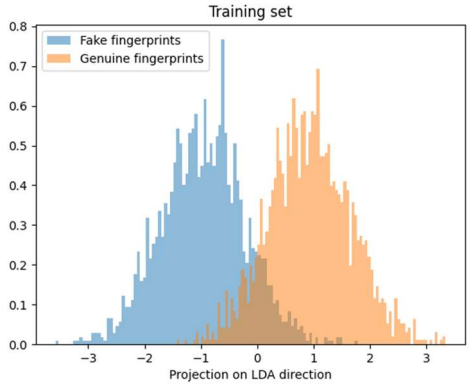


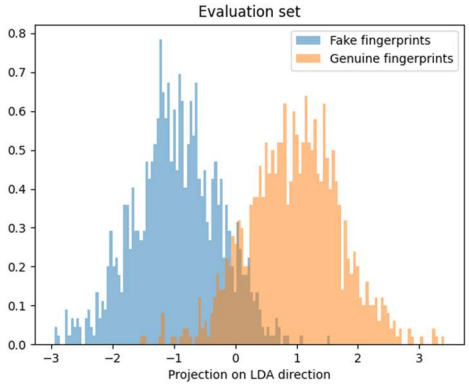
With LDA we can notice that the resulting dataset projection does not differ too much from the dataset projection over the PCA direction with the larger variance, generating two class distributions with separated peaks but with an intermediate overlapping area around the origin.

LDA is particularly useful for classification purposes. In order to perform a classification, we need:

- a *classifier*;
- a *training set*, over which we train the classifier;
- a *validation set*, over which we evaluate the performance classifier.

The last two points can be obtained by splitting the original dataset into a **training set** (consisting of $\frac{2}{3}$ of the total samples) and a **validation set** (consisting of $\frac{1}{3}$ of the total samples).
The projections of the training set and the validation set over the LDA direction are the following:





These two plots look quite similar to the plot of the overall dataset, implying that the dataset has a low internal variability, making overfitting a minor issue.
We then choose a simple **classifier** based on a threshold defined as

$$t = \frac{\mu_1 + \mu_0}{2}$$

where μ_1 is the mean of genuine samples and μ_0 is the mean of fake samples as calculated from the training set. Then, we assign a label to a test sample x_t (included in the validation set) with the following rule:

$$\begin{cases} h_0 \text{ if } x_t < t \\ h_1 \text{ if } x_t \geq t \end{cases}$$

Where the label h_0 correspond to a fake fingerprint and h_1 correspond to a genuine fingerprint.
By applying this model to the whole validation set, we obtain the following results:

Threshold $\boldsymbol{t}$	-0.018534376786207174
Total validation samples	2000
Total classification errors	186
Error rate	**9.30%**

If we try to modify the value of the threshold we can notice that the error rate seems to be slightly reduced up to 9.00% when selecting $t' \cong -0.012$. Selecting a threshold greater than t' or lower than t will result in a worse error rate, and same thing seems to happen in the range of values $t < x < t'$, but, apart from this, even if the thresholds t and t' are very close, this difference is enough for influencing the classification.

2.3 LDA + PCA as pre-processing

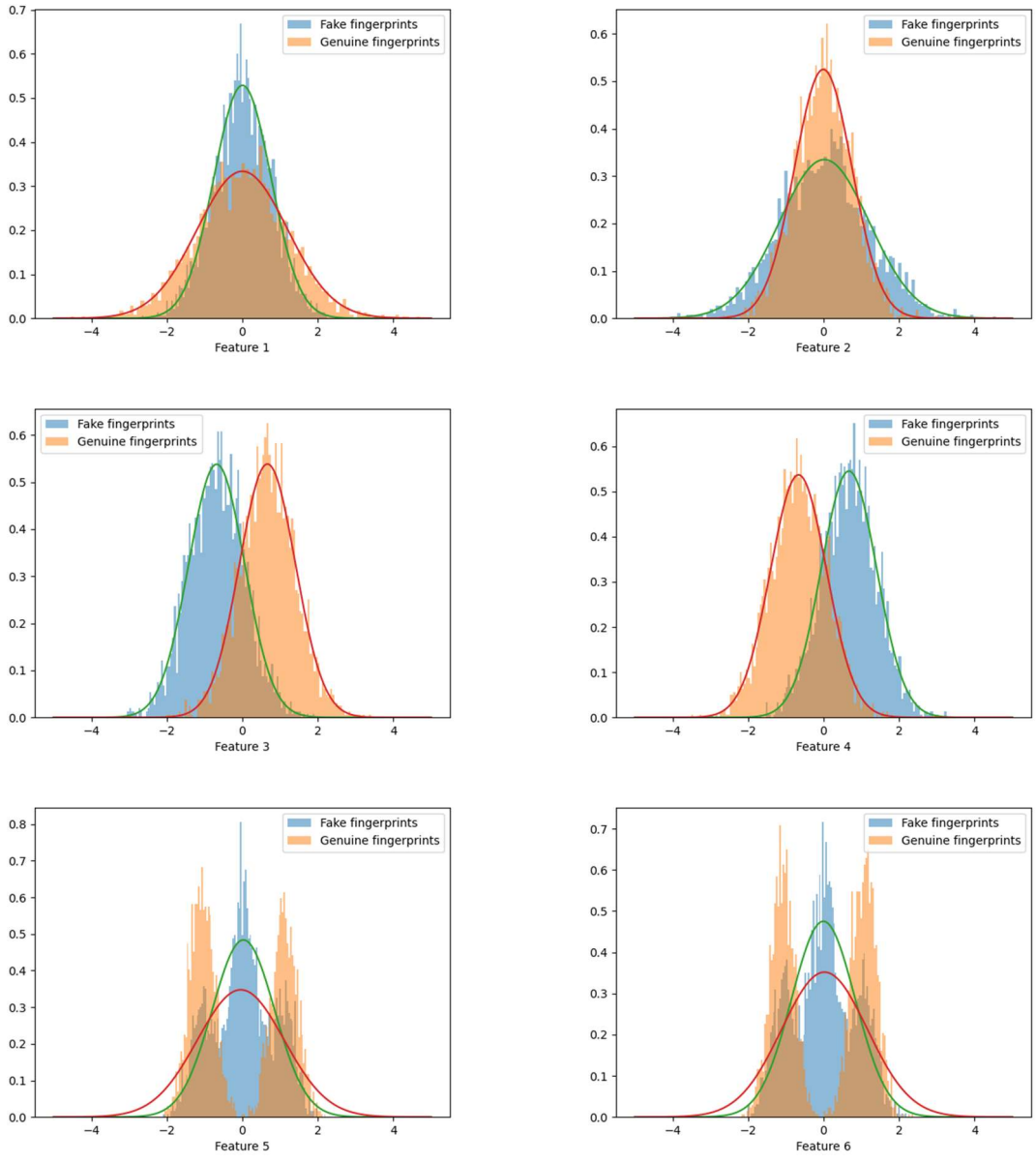
A more complete approach consists in combining PCA and LDA by using PCA as a pre-processing technique for reducing the dimensionality of the training set and then using LDA as a classification tool as shown before. Of course, we can apply PCA with different values of the final dimension m, in this case from $m = 6$ (no dimensionality reduction) to $m = 1$ (maximum dimensionality reduction). This approach produces the following final results concerning the LDA classification:

PCA dimensions	**Threshold t**	**Misclassifications**	**Error rate**
$m = 6$	-0.018534376786207285	186	9.30%
$m = 5$	-0.018519178157787586	186	9.30%
$m = 4$	-0.01831713244834865	185	**9.25%**
$m = 3$	-0.018398189889104022	185	**9.25%**
$m = 2$	-0.01828175332678572	185	**9.25%**
$m = 1$	-0.01759903023797671	187	9.35%

This table shows that applying PCA as a pre-processing strategy, with this specific dataset and with this specific classifier, is able in some cases to make the classification slightly more accurate. In particular, we can observe a decrease of the error rate by 0.05% by reducing the dimensionality to a value $m \in [2,3,4]$. On the contrary, the error rate remains the same when reducing to $m \in [5,6]$ and it also get slightly worse by reducing to $m = 1$, where we probably lose some useful discriminant information.

3. Gaussian generative models

A more robust classification model that we can apply to the problem is a generative model, specifically the Multivariate Gaussian Classifier, for which we assume that the class-conditional probability $f_{X|C}(x|c)$ of each class c can be approximated with a gaussian distribution $N(x; \mu_c, \Sigma_c)$.
In order to verify the goodness of this assumption for our dataset, we can compare each class likelihood univariate histogram with the corresponding univariate gaussian estimation.
The estimation is based, for each class c and for each feature j, on the parameters $\mu_{[j],c}$ and $\sigma^2_{[j],c}$, which can be retrieved with the related ML estimators.
The plots resulting from the gaussian fitting are the following:



From these plots we can notice that the gaussian fitting does not have the same goodness for all the features.

In particular:

- The gaussian fitting seems to be particularly effective for both classes in features f_1, f_2, f_3 and f_4, where the shape of the histograms and the one of the related gaussian estimations are quite similar. Of course the approximation is not exact since the histograms show a greater probability density over the peaks than the one of the related gaussian.
- On the other hand, the gaussian fitting is much less accurate with both classes of features f_5 and f_6. This is clear since, even if the gaussian estimation has the same mean and variance of the related histograms, it doesn't follow its probability density distribution, which is much more discontinuous.
  For example, both features f_5 and f_6 of genuine samples have a null probability density over the mean, which, on the contrary, is the greater probability density area of the related gaussian estimation. This discrepancies produce an overall bad approximation.

3.1 Multivariate Gaussian model (MVG)

The first gaussian classification model we apply is the default Multivariate Gaussian classifier (MVG), for which we model the likelihood distribution $f_{X|C}(x|c)$ as a normal distribution $N(x; \mu_c, \Sigma_c)$, where μ_c and Σ_c are the mean and covariance matrix related to class c. Then we can use this PDF to evaluate the optimal Bayes decision for a test sample x_t by calculating the related log-posterior ratio:

$$log\left(\frac{P(C=h_1|x_t)}{P(C=h_0|x_t)}\right) = log\left(\frac{f_{X|C}(x_t|h_1)}{f_{X|C}(x_t|h_0)}\right) + log\left(\frac{\pi}{1-\pi}\right)$$

Since we assume that the priors are $P(C=h_1) = P(C=h_0) = \frac{1}{2}$ the prior-dependent term is null and we can classify the samples only according to the log-likelihood ratio, that is

$$log\left(\frac{P(C=h_1|x_t)}{P(C=h_0|x_t)}\right) = llr(x_t) = log\left(\frac{f_{X|C}(x_t|h_1)}{f_{X|C}(x_t|h_0)}\right)$$

In particular:

- we assign class h_1 (*genuine fingerprint*) if $llr(x_t) \geq 0$
- we assign class h_0 (*fake fingerprint*) if $llr(x_t) < 0$

By applying this model to the evaluation set defined in the previous chapter, we obtain the following results:

Total validation samples	2000
Total classification errors	140
Error rate	**7.00%**

3.2 Gaussian model with tied covariances

We now try to apply the tied gaussian model, which assumes that all the sample clusters are spread in the same way around their mean, that is, they have the same covariance matrix. In this case we model the likelihood $f_{X|C}(x|c)$ as the distribution $N(x; \mu_C, \Sigma)$, where the covariance matrix Σ does not depend on the class c.
The results obtained by this model on the evaluation set are the following:

Total validation samples	2000
Total classification errors	186
Error rate	**9.30%**

This model (9.30% error rate) seems to be as accurate as for LDA (9.30% error rate) but also inaccurate with respect to the MVG model (7.00% error rate). This can be related to the fact that, as seen in the plots of chapter 1, the difference in the spread of the clusters of the various classes is significant enough to not justify the tied gaussian model assumption.

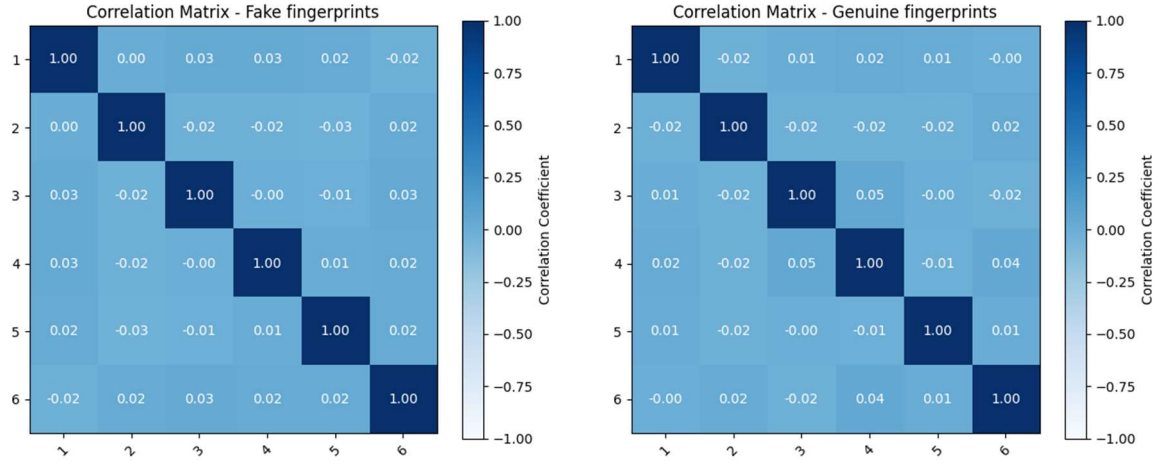
3.3 Naive Gaussian model

The naive gaussian model assumes uncorrelation between the features of each sample. In this case the gaussian PDF $f_{X|C}(x|c) \sim N(x; \mu_C, \Sigma_C)$ is characterized by a covariance matrix Σ_c that is diagonal. We can adapt the previous model by simply diagonalizing Σ_c and using it for evaluating $llr(x_t)$ for a generic test sample x_t.
The results obtained on the evaluation set are the following:

Total validation samples	2000
Total classification errors	144
Error rate	**7.20%**

In this case the naive gaussian model is slightly less accurate than the multivariate gaussian model but it's still more accurate than the tied gaussian model. This proves that the features of the dataset have a relatively small correlation.

In fact, in order to better visualize the correlation among the dataset features we can calculate the Pearson's correlation matrix related to each class.
The resulting plots are the following:



These plots clearly show what we already noticed by applying the naive gaussian model, that is, the features are weekly correlated and this allows us to consider the model assumption as essentially correct.

3.4 Gaussian models goodness evaluation

At the beginning of this chapter we tried to fit a gaussian distribution over each one of the features for each class. This showed that the last two features, both for classes h_0 and h_1, have a bad fitting, since the related histograms clearly do not show a gaussian-like shape.
This fitting inaccuracy could probably affect negatively the previous classifications and, in order to verify this, we can repeat the three gaussian classifications excluding the features f_5 and f_6.

This approach leads to the following results:

Model	Classification errors	Error rate
Multivariate gaussian	159	**7.95%**
Tied gaussian	190	**9.50%**
Naive Bayes gaussian	153	**7.65%**

So, after excluding the last two features, we get an overall worsening of all the classification models, but this time the naive gaussian model is the one performing better, having a lower error rate than the other two classifiers.
This results mean two things:

- removing features f_5 and f_6, that is, the only features that didn't show a good gaussian fitting, strengthens the naive gaussian assumption for the remaining features, making this model the most accurate one among the three;
- the fact that the accuracy of all the three models got slightly worsened suggests that the features f_5 and f_6 were retaining some class-discriminant information that, once removed, made the gaussian models more accurate on the other features but not on the overall dataset; in fact, the calculation of the error rates was still based on the labels of the original (complete) dataset.

Moreover, given that the pairs of features f_1, f_2 and f_3, f_4 show different distributions (see chapter 1), we can observe the behavior of these features (jointly) with respect to the previous three gaussian models.
For features f_1 and f_2 we obtain the following results:

Model	Classification errors	Error rate
Multivariate gaussian	730	**36.50%**
Tied gaussian	989	**49.45%**
Naive Bayes gaussian	726	**36.30%**

In this case we can notice that the large overlapping between the distributions of f_1 and f_2 makes classification much more difficult and, as a result, it heavily worsens all the error rates. Besides, the class-conditional distributions of both the features have a good gaussian fitting and, because of this, naive gaussian is still the model with the best results.
On the other hand, given that the distributions of different classes for the same feature have a different variance, the assumption of the tied gaussian model (same spread among classes) is inaccurate and gives bad results.

In the same way, for features f_3 and f_4 we obtain the following results:

Model	Classification errors	Error rate
Multivariate gaussian	189	**9.45%**
Tied gaussian	188	**9.40%**
Naive Bayes gaussian	189	**9.45%**

In this case we can notice that considering only features f_3 and f_4 does not affect the classification in a significant way. This means that the most part of the class-discriminant information is retained by these two features. Moreover, given that the distributions of different classes have almost the same variance, the tied gaussian assumption is verified and the related model is, in fact, the one performing best. The weak correlation among f_3 and f_4 and the good gaussian fitting make the other two models just slightly less efficient but still competitive.

3.5 Classification with PCA as pre-processing

Finally, we can try to use PCA as a strategy for pre-processing both the training set and the validation set before applying the gaussian classification models. Of course, the dimensionality m of the dataset can be reduced in the range $1 \leq m \leq 6$, and this can eventually lead to different results. The error rates related to each gaussian model applied after a PCA pre-processing with final dimensionality m can be found in the following table:

PCA dimension	Multivariate gaussian	Tied gaussian	Naive gaussian
1	9.25%	9.35%	9.25%
2	8.80%	9.25%	8.85%
3	8.80%	9.25%	9.00%
4	8.05%	9.25%	8.85%
5	7.10%	9.30%	8.75%
6	**7.00%**	9.30%	8.90%

As we can see, PCA leads to worse or equal results for both multivariate gaussian and naive gaussian, while for tied gaussian, by using $m \in [2,4]$, we get slightly better results (9.25% error rate vs. 9.30% without PCA). The overall best result (MVG model with $m = 6$) has a 7.00% error rate, which is the same as the one without PCA; in fact, in this case, since $m = 6$, there is no actual dimensionality reduction but the dataset gets transformed according to the principal directions, which does not affect the model performance.
On the other hand we can say that, given that the worsening in the error rates is not too high (even for low values of m), applying PCA for pre-processing can be a good compromise if we need to reduce the dimensionality of the dataset.

4. Bayes decisions and model evaluation

In this chapter we try to analyze our models according to the Bayes risk and introducing costs related to different types of mis-classifications.
In our case, a generic application, over which we employ our models, can be represented by a triplet (π_1, C_{fn}, C_{fp}), where π_1 is the prior probability of genuine fingerprints, C_{fn} is the cost of refusing genuine samples and C_{fp} is the cost of accepting fake samples.

We then consider the following triplets:

- $(0.5, 1.0, 1.0)$: application with uniform prior and uniform costs (same probability of having a true or a fake sample and same costs for the two types of mis-classifications).
- $(0.9, 1.0, 1.0)$: application with non-uniform prior and uniform costs (there is an higher probability of true samples but there are same costs for the two types of mis-classifications).
- $(0.1, 1.0, 1.0)$: application with non-uniform prior and uniform costs (there is an higher probability of fake samples but there are same costs for the two types of mis-classifications).
- $(0.5, 1.0, 9.0)$: application with uniform prior and non-uniform costs (same probability of having a true or a fake sample but accepting a fake sample has a higher cost).
- $(0.5, 9.0, 1.0)$: application with uniform prior and non-uniform costs (same probability of having a true or a fake samples but refusing a true sample has a higher cost).

We can also express a generic triplet (π_1, C_{fn}, C_{fp}) in terms of an effective prior $\tilde{\pi}$, that is such that the applications (π_1, C_{fn}, C_{fp}) and $(\tilde{\pi}, 1, 1)$ are the same, that is, they have the same Bayes risk.
The effective prior can be calculated as

$$\tilde{\pi} = \frac{\pi_1 C_{fn}}{\pi_1 C_{fn} + (1 - \pi_1) C_{fp}}$$

In this case the first three triplets are already expressed in terms of effective priors, which are $\tilde{\pi} = 0.5$, $\tilde{\pi} = 0.9$ and $\tilde{\pi} = 0.1$.
The fourth triplet has an effective prior $\tilde{\pi} = 0.1$ and the last one has an effective prior $\tilde{\pi} = 0.9$. This means that $(0.5, 1.0, 9.0)$ and $(0.1, 1.0, 1.0)$ are the same application, as well as $(0.5, 9.0, 1.0)$ and $(0.9, 1.0, 1.0)$. This reflects the fact that an higher cost for accepting a fake sample actually corresponds to a low prior probability of a genuine sample, and vice versa.

4.1 Application (0.5, 1.0, 1.0)

For the application (0.5, 1.0, 1.0) we can compute the actual DCF and the minimum DCF related to the three gaussian models employed in the previous chapter (with and without the PCA pre-processing).
For each model we define the actual (normalized) DCF as

$$DCF = \frac{DCF_u}{\min(\pi_1 C_{fn}, (1 - \pi_1) C_{fp})} = \frac{\pi_1 C_{fn} P_{fn} + (1 - \pi_1) C_{fp} P_{fp}}{\min(\pi_1 C_{fn}, (1 - \pi_1) C_{fp})}$$

The minimum DCF is calculated by moving the application threshold $t = -log \frac{\tilde{\pi}}{1-\tilde{\pi}}$ among the values of the scores represented by the log-likelihood ratios.

The obtained values are in the following table:

PCA	**Multivariate Gaussian**		**Tied Gaussian**		**Naive Gaussian**	
	actDCF	*minDCF*	*actDCF*	*minDCF*	*actDCF*	*minDCF*
No PCA	**0.140**	**0.130**	0.186	0.181	0.144	0.131
$m = 6$	0.140	0.130	0.186	0.181	0.178	0.173
$m = 5$	0.142	0.133	0.186	0.181	0.175	0.174
$m = 4$	0.161	0.154	0.185	0.183	0.177	0.173
$m = 3$	0.176	0.173	0.185	0.183	0.180	0.176
$m = 2$	0.176	0.173	0.185	0.179	0.177	0.171
$m = 1$	0.185	0.177	0.187	0.177	0.185	0.177

From the table we can see that the best minimum DCF is the one related to the MVG model applied and evaluated over the dataset without PCA pre-processing ($minDCF = 0.130$). This also stands for the best actual DCF ($actDCF = 0.140$).
Moreover, the employed models seems to be quite well calibrated since the difference between each *actDCF* and the related *minDCF* is always lower than 10% and almost always lower than 5%.
In particular, for this specific application, the models have a similar level of calibration, which is quite homogeneous among the different PCA pre-processing dimensionalities.

4.2 Application (0.9, 1.0, 1.0)

The values of the actual and minimum DCF of the application (0.9, 1.0, 1.0) related to the three gaussian models (with and without PCA pre-processing) are the following:

PCA	**Multivariate Gaussian**		**Tied Gaussian**		**Naive Gaussian**	
	actDCF	*minDCF*	*actDCF*	*minDCF*	*actDCF*	*minDCF*
No PCA	0.400	**0.342**	0.463	0.442	0.389	0.352
$m = 6$	0.400	0.342	0.463	0.442	0.451	0.437
$m = 5$	**0.398**	0.351	0.462	0.445	0.466	0.437
$m = 4$	0.460	0.415	0.462	0.446	0.463	0.431
$m = 3$	0.468	0.439	0.457	0.435	0.459	0.434
$m = 2$	0.443	0.438	0.479	0.436	0.442	0.432
$m = 1$	0.478	0.434	0.481	0.435	0.478	0.434

From the table we can see that both the best minimum DCF ($minDCF = 0.342$) and the best actual DCF ($actDCF = 0.398$) are associated with the MVG model; one with $m = 5$ and the other without dimensionality reduction.
In this case the models have a slightly worse calibration, since some actual DCF have a difference with the related minimum DCF which is over 10% (like for the MVG model with no PCA or with $m = 5$).
Also in this case, the models have a similar calibration level, which is again not homogeneous.

4.3 Application (0.1, 1.0, 1.0)

The values of the actual and minimum DCF of the application (0.1, 1.0, 1.0) related to the three gaussian models (with and without PCA pre-processing) are the following:

PCA	**Multivariate Gaussian**		**Tied Gaussian**		**Naive Gaussian**	
	actDCF	*minDCF*	*actDCF*	*minDCF*	*actDCF*	*minDCF*
No PCA	0.305	0.263	0.406	0.363	**0.302**	**0.259**
$m = 6$	0.305	0.263	0.406	0.363	0.392	0.353

$m = 5$	0.304	0.274	0.405	0.364	0.393	0.354
$m = 4$	0.353	0.301	0.403	0.363	0.397	0.361
$m = 3$	0.388	0.356	0.408	0.370	0.395	0.366
$m = 2$	0.388	0.353	0.396	0.363	0.387	0.356
$m = 1$	0.397	0.386	0.402	0.370	0.397	0.369

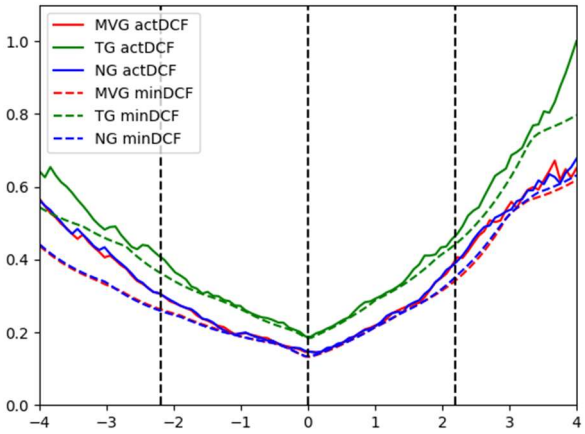
From the table we can see that both the best minimum DCF ($minDCF = 0.259$) and the best actual DCF ($actDCF = 0.302$) are associated with the naive gaussian model with no PCA pre-processing.
The calibration seems to be similar (in percentage) to the one of the previous application, so it's also slightly worse than the one of the first application.
Lastly, the calibration level among the different models doesn't show significant variations.

4.4 Bayes error plot

In order to better understand the results that we obtained so far, we consider the application with $\tilde{\pi} = 0.1$ and the gaussian models with no PCA pre-processing and we plot both the actual DCF and the minimum DCF with respect to a moving effective prior $\tilde{\pi}$.
The x-axis represents the value of $log\frac{\tilde{\pi}}{1-\tilde{\pi}}$, which is the opposite of the threshold $t = -log\frac{\tilde{\pi}}{1-\tilde{\pi}}$ used for the classification decision rule.
The y-axis represents both $actDCF$ and $minDCF$ related to the specific value of $\tilde{\pi}$ and for all the gaussian models.
In this case, since we are considering $\tilde{\pi} = 0.1$, we have that

$$log\frac{\tilde{\pi}}{1-\tilde{\pi}} \cong -2.20$$

This is the value of the x-axis over which we can evaluate the performance of our models on the chosen application.
The resulting plot is the following:



The plot shows how the $actDCF$ and $minDCF$ of a same model have a similar "shape" and are close to each other, meaning that, over the considered interval, the models are both consistent and quite well calibrated.
The intersections between the plots and the line $x = -2.20$ correspond to the values of $actDCF$ and $minDCF$ that can be found in the previous table for the dataset with no PCA.
The other two lines (with equations $x = 0$ and $x = 2.20$) are the ones related to the other two applications ($\tilde{\pi} = 0.5$ and $\tilde{\pi} = 0.9$).

5. Logistic regression model

We now analyze the effectiveness of the logistic regression model employed on this classification task. We start considering the standard non prior-weighted version of the model, for which we classify a sample x_i using the rule $s_i = w^T x_i + b \gtrless 0$ and for which:

$$P(C = h_1|x, w, b) = \frac{1}{1 + e^{-(w^T x + b)}} = \sigma(w^T x + b) = \sigma\big(s(x)\big)$$

where the model parameters (w, b) are obtained as follows:

$$w^*, b^* = \arg\min_{w,b} \frac{\lambda}{2}||w||^2 + \frac{1}{n}\sum_{i=1}^{n} l(z_i s_i)$$

In general, for a sample x_i, the value $s_i = w^T x_i + b$ is already sufficient to provide a score (without a strict probabilistic interpretation since s_i can be lower than 0 and greater than 1), such that

- we assign h_1 to x_i if $s_i \geq 0$ (the sample is above the hyperplane $w^T x + b = 0$)
- we assign h_0 to x_i if $s_i < 0$ (the sample is below the hyperplane $w^T x + b = 0$)

5.1 Non prior-weighted model employment

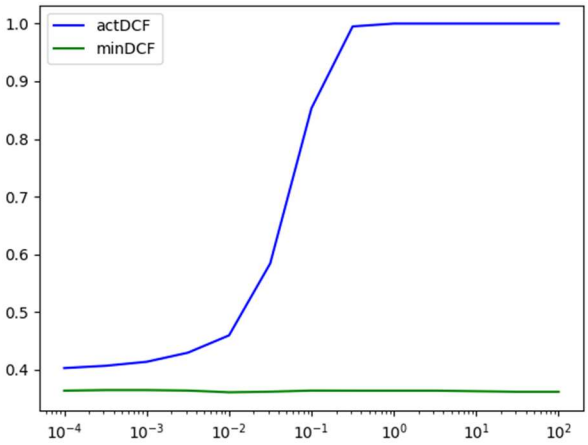
We consider again the application $(\tilde{\pi}, 1, 1)$ where $\tilde{\pi} = 0.1$.
If we train the logistic regression model, for different values of λ, and we apply the previous classification rule over the validation set we obtain the following results:

λ	Error rate	actDCF	minDCF
$1.0 \cdot 10^{-4}$	9.30%	**0.403**	0.364
$3.2 \cdot 10^{-4}$	9.35%	0.407	0.365
$1.0 \cdot 10^{-3}$	9.35%	0.414	0.365
$3.2 \cdot 10^{-3}$	9.25%	0.430	0.364
$1.0 \cdot 10^{-2}$	9.25%	0.460	**0.361**
$3.2 \cdot 10^{-2}$	9.20%	0.584	0.362
$1.0 \cdot 10^{-1}$	9.25%	0.853	0.364
$3.2 \cdot 10^{-1}$	9.30%	0.995	0.364
1.0	9.25%	1.000	0.364
3.2	9.25%	1.000	0.364
$1.0 \cdot 10^{1}$	9.50%	1.000	0.363
$3.2 \cdot 10^{1}$	9.60%	1.000	0.362
$1.0 \cdot 10^{2}$	12.85%	1.000	0.362

The previous table shows that the error rates have negligible variations when $\lambda < 3.2$, after which the regularization term becomes too high, causing the minimization algorithm to provide small values of $||w||$, which correspond to a worse classification since the samples are "closer" to the linear decision boundary $w^T x + b = 0$.

In order to better understand the data we can also plot the actual DCF and the minimum DCF as a function of λ.

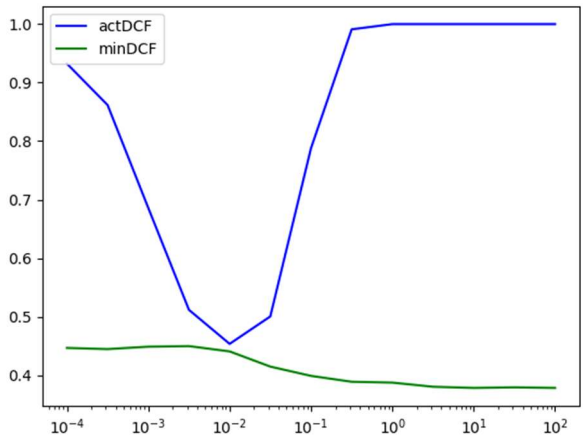
The resulting plot is the following:



The plot shows how the model has lower values of the actual DCF (actDCF < 0.500) when $\lambda \leq 10^{-2}$, after which it grows faster until $\lambda \leq 3.2 \cdot 10^{-1}$, after which it stabilizes to the maximum value of 1, meaning that, for the application $(0.1, 1, 1)$, the classifier performs as poorly as possible. This is also influenced by the fact that high values of the regularization term λ cause a loss in the model's ability to correctly classify the samples.

The minimum DCF, on the other hand, follows a different evolution, being stable around the value 0.364 with a negligible decrease for λ in the order of 10^{-2} and for $\lambda \geq 10^{1}$.
This difference shows that training the model considering the prior of the training set (which is different to the one of the validation set) has a negative impact on the choice of a good threshold, making the model badly calibrated.

In general, the purpose of the regularization term is to avoid overfitting. This issue may be less influent for large datasets, making the contribution of λ uneffective.
To analyze this phenomenon we try to emply the model over a small portion of the training set (we keep just 1 out of 50 training samples), for which overfitting is a more impacting issue.
The resulting plot is the following:



The plot shows that for low values of λ ($\lambda < 10^{-2}$) overfitting becomes a major issue.
For $\lambda = 10^{-2}$ we obtain a balanced situation of low overfitting and good calibration. On the other hand, when λ grows too much, its impact is the same of the one in the previous case, but in this case the major risk is not overfitting but the opposite phenomenon, that is, bad classification, caused by values of $||w||$ that are too small.

5.2 Prior-weighted model employment

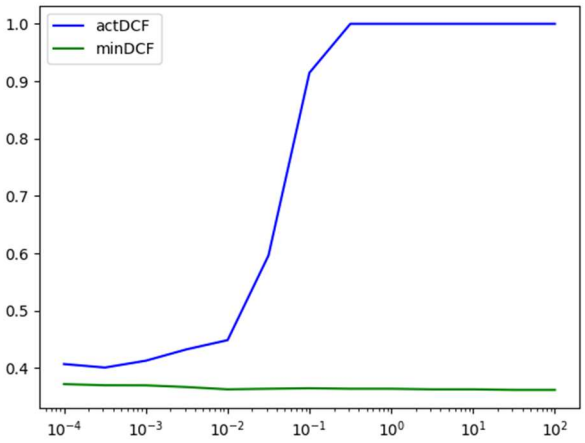
We now try to employ the prior-weighted model, for which we estimate (w, b) such that they minimize the logistic loss expressed as

$$(w^*, b^*) = \arg\min_{w,b} \frac{\lambda}{2}||w||^2 + \frac{\pi_T}{n_T} \sum_{i|z_i=1} l(z_i s_i) + \frac{1-\pi_T}{n_F} \sum_{i|z_i=-1} l(z_i s_i)$$

where π_T is the application prior related to class h_1 (true fingerprints). In this case we use the effective prior $\tilde{\pi} = 0.1$.
The purpose of this variant is to to adapt the model to the application prior (if we already know it in advance), in order to align the results to the data over which the model will be employed.
The obtained results can be found in the following table:

λ	Error rate	actDCF	minDCF
$1.0 \cdot 10^{-4}$	16.85%	0.407	0.372
$3.2 \cdot 10^{-4}$	16.95%	**0.401**	0.370
$1.0 \cdot 10^{-3}$	17.55%	0.413	0.370
$3.2 \cdot 10^{-3}$	18.55%	0.433	0.367
$1.0 \cdot 10^{-2}$	21.80%	0.449	0.363
$3.2 \cdot 10^{-2}$	29.65%	0.596	0.364
$1.0 \cdot 10^{-1}$	46.10%	0.914	0.365
$3.2 \cdot 10^{-1}$	50.40%	1.000	0.364
1.0	50.40%	1.000	0.364
3.2	50.40%	1.000	0.363
$1.0 \cdot 10^{1}$	50.40%	1.000	0.363
$3.2 \cdot 10^{1}$	50.40%	1.000	**0.362**
$1.0 \cdot 10^{2}$	50.40%	1.000	**0.362**

Once again we can plot the actual DCF and the minimum DCF as functions of the regularization term λ. The resulting plot is the following:



Both the table and the plot show that the prior-weighted logistic regression is not able to improve the DCF in a significan way for any value of λ. On the contrary, both the actual DCF and the minimum DCF remans quite the same even with a major worsening of the related error rates (which do not reflect the overall cost of the classifications).

This means that using this varaint of the logistic regression, for this specific application, does not provide any advantage that could make it a better choice over the standard logistic regression model.

5.3 Quadratic logistic regression

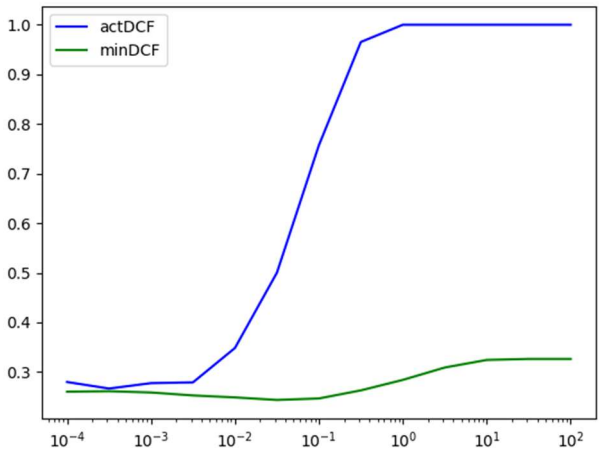
We now try to apply the non prior-weighted logistic regression in its quadratic form. This chan be achieved by expanding the feature space applying to each sample x the following transformation:

$$\Phi(x) = \begin{bmatrix} \text{vec}(xx^T) \\ x \end{bmatrix}$$

This allows us to train the standard logistic regression model over the resulting space and, therefore, to express the classification scores as $s(x, w, c) = w^T \Phi(x) + c$. By doing this we are considering a linear decision boundary over the space mapped by Φ, which actually corresponds to a quadratic decision boundary over the original space.
By applying this model we obtain the following results:

λ	Error rate	actDCF	minDCF
$1.0 \cdot 10^{-4}$	6.05%	0.280	0.260
$3.2 \cdot 10^{-4}$	6.05%	**0.267**	0.261
$1.0 \cdot 10^{-3}$	5.95%	0.278	0.259
$3.2 \cdot 10^{-3}$	5.90%	0.280	0.253
$1.0 \cdot 10^{-2}$	5.90%	0.348	0.249
$3.2 \cdot 10^{-2}$	5.90%	0.500	**0.244**
$1.0 \cdot 10^{-1}$	6.05%	0.757	0.247
$3.2 \cdot 10^{-1}$	6.10%	0.965	0.263
1.0	6.40%	1.000	0.284
3.2	7.10%	1.000	0.309
$1.0 \cdot 10^{1}$	7.10%	1.000	0.324
$3.2 \cdot 10^{1}$	7.35%	1.000	0.326
$1.0 \cdot 10^{2}$	9.30%	1.000	0.326

The related plot of the actual DCF and the minimum DCF as functions of λ is the following:

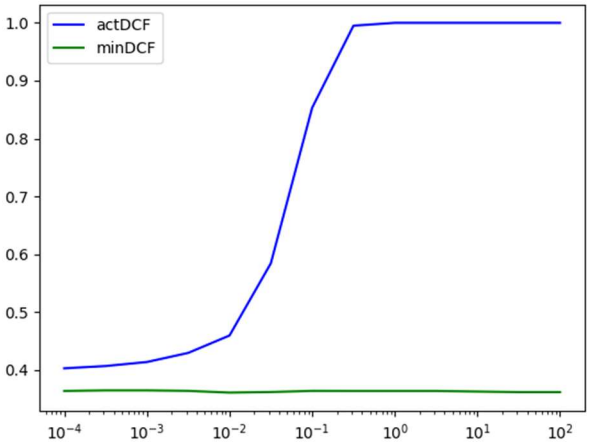


These results show that the quadratic logistic regression is able to produce better results.

In particular, the regularization term λ is more effective since, when it has a sufficiently low value, the miscalibration is partially reduced with respect to the linear logistic regression models (both the prior-weighted and the non prior-weighted ones), also producing smaller values of both the actual DCF and the minimum DCF.
This also happens when λ is too high, that is, when the classifier has a poor performance. In this case the minimum DCF is slightly increased, making it closer to the related actual DCF, which is instead not affected by this model.
If we do not consider the misclassification costs we can also observe that the quadratic logistic regression model is able to reduce the error rates for all the values of λ.

5.4 Effects of data centering

Since the regularized model is not invariant to affine transformations, we now analyze the effect of centering the traning set and the validation set with respect to the mean of the training set.
After applying the regularized linear logistic regression model to the transformed data, we obtain the following plot:



The plot shows how, even with the regularized model, the data transformation has actually no significant effect on the DCFs. This is because the traning dataset is already standardized, meaning that it's already almost centered. In fact, the mean vector related to the training set results to be the following:

```
[-0.00561385  0.00333033 -0.00840402  0.01009222 -0.01414677  0.00585712]
```

This means that centering both the training and the validation set produce a neglible transformation, which is not able to modify the model results in an impacting way.

5.5 Models performance recap

Up to now we have tried both the gaussian generative models and the discriminative logistic regression models, and we have evaluated them with respect to the metric represented by the actual DCF and the minimum DCF.
The following table sums up the results (only related to the minimum DCF) we obtained so far for the application with $\tilde{\pi} = 0.1$:

Model	**Model assumption**	**Best *minDCF***
Multivariate gaussian	The likelihood distribution $f_{X\|C}(x\|c)$ behaves like a gaussian distribution $N(x; \mu_C, \Sigma_C)$	0.263

Tied gaussian	The covariance matrices of the gaussian distributions witch model the likelihood distributions $f_{X\|C}(x\|c)$ don't depend on the class c	0.363
Naive gaussian	All the features of the samples are statistically independent and behave like a univariate gaussian distribution.	0.259
Linear logistic regression (*non prior-weighted*)	The log-posterior probabilities of the two classes have a linear relationship	0.361
Linear logistic regression (*prior-weighted*)	The log-posterior probabilities of the two classes have a linear relationship	0.362
Quadratic logistic regression (*non prior-weighted*)	The log-posterior probabilities of the two classes have a quadratic relationship	0.244

Note: the previous table takes into account only the best minimum DCF among the ones obtained with different PCA pre-processings (for the gaussian models) or with different values of λ (for the logistic regression models).

So far, the model that has achieved the best minimum DCF (minDCF $= 0.244$) is the quadratic non prior-weighted logistic regression model (with a regularization term $\lambda = 3.2 \cdot 10^{-2}$).
Moreover, the models that have as a consequence or as an assumption the linearity of the log-posterior ratio, that is, the tied gaussian model and the linear logistic regression model, have the worst minimum DCF. On the other hand, the model which implies a quadratic relationship among the log-posterior probabilities, that is, the naive gaussian model, has a minimum DCF which is close to the one of the quadratic logistic regression model. The same goes with the result obtained with the multivariate gaussian model.

6. Support Vector Machines (SVM)

6.1 Linear SVM

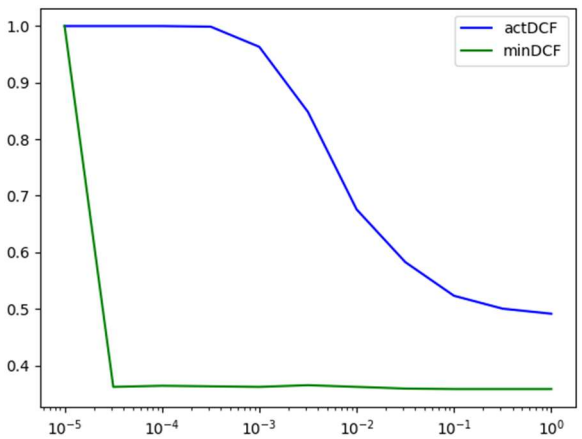
We now try to perform classifications using the linear SVM model. Like for logistic regression, this model is discriminative and allows for linear and non-linear classifications.
The primary assumption of SVM for linearly separable classes is that there exists a decision surface $w^T x + b = 0$ that maximizes the margin of the closest point with respect to the surface itself.
The model parameters (w, b) can be indirectly calculated by solving the SVM dual formulation with respect to α, that is

$$\arg\max_\alpha \hat{J}_D(\alpha) = \arg\max_\alpha \left(-\frac{1}{2}\alpha^T \hat{H}\alpha + \alpha^T 1\right)$$

where $\hat{H}i_j = z_i z_j \hat{x}_i^T \hat{x}j$ and where the original space x was transformed into a space $\hat{x}$ that allows mitigating the effect of the regularization of the bias term b. The model also defines a constant C that acts as a trade-off between the model generalization and the misclassification acceptance.
By applying this model to our dataset with different values of C we obtain the following results:

C	Error rate	actDCF	minDCF
$1.0 \cdot 10^{-5}$	50.40%	1.000	1.000
$3.2 \cdot 10^{-5}$	9.15%	1.000	0.362
$1.0 \cdot 10^{-4}$	9.20%	1.000	0.364
$3.2 \cdot 10^{-4}$	9.25%	0.999	0.362
$1.0 \cdot 10^{-3}$	9.30%	0.963	0.362
$3.2 \cdot 10^{-3}$	9.35%	0.848	0.365
$1.0 \cdot 10^{-2}$	9.25%	0.677	0.362
$3.2 \cdot 10^{-2}$	9.15%	0.583	0.359
$1.0 \cdot 10^{-1}$	9.15%	0.523	**0.358**
$3.2 \cdot 10^{-1}$	9.10%	0.501	**0.358**
1.0	9.05%	**0.491**	**0.358**

As always, we can also plot both the actual DCF and the minimum DCF as a function of C.



Both the table and the plot show that, excluding the smallest value, the variation of C does not influence the error rate in a significant way. On the contrary, the actual DCF has an important variation and decreases (starting from the maximum value of 1.0) as C increases, that is, as the model loses generalization and focuses on containing the misclassifications. The minimum DCF has its maximum value for $C = 1.0 \cdot 10^{-5}$ and then, as C increases, it drops to much lower values maintaining itself stable on a close range of values. This means that too low values of C have the consequence of accepting too many errors, increasing the cost of the model up to a value that makes its performance the poorest possible (with respect to both the actual DCF and the minimum DCF).
Since the score produced by the SVM model have no probabilistic interpretation, the difference between the actual DCF and the minimum DCF is quite high, implyng a bad calibration. Anyway, the level of miscalibration is lower for high values of C.

Overall, this model has a performance which is similar to the one of the other linear models, that is, the linear logistic regression and the tied gaussian model. In particular, both the error rate and the minimum DCF are in the same order of magnitude.

Being that SVM is not invariant to affine transformations, we may need to pre-process the data, for example centering both the training and the evaluation set with respect to the mean of the training set. Anyway, this pre-processing strategy does not change the performance of the model; the variations in the error rates is negligible and the DCF plot is essentially the same as the one showed in the non-centered case. This, like for the logistic regression case, is because the training set is already standardized.

6.2 SVM with polynomial kernel

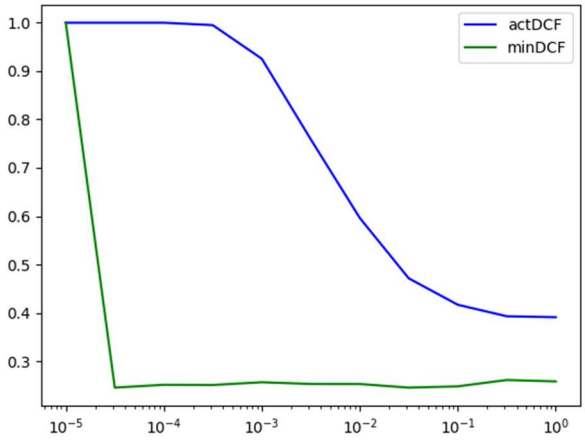
If we want to use SVM to perform non-linear classifications, we can exploit the kernel functions, which allow to solve the dual formulation of the model with respect to a specific feature expansion but without explicitly calculating it. This can be made because the dual formulation does not depend directly on the training data but on dot-products of pair of samples.
We first use the polynomial kernel, which has the following equation:

$$k(x_1, x_2) = (x_1^T x_2 + c)^d$$

where d is the degree of the decision hyperplane.
In this case we consider $d = 2$ (so we have quadratic decision surfaces) and $c = 1$.
By applying this model to our dataset with different values of C we obtain the following results:

C	Error rate	actDCF	minDCF
$1.0 \cdot 10^{-5}$	50.40%	1.000	1.000
$3.2 \cdot 10^{-5}$	7.80%	1.000	**0.245**
$1.0 \cdot 10^{-4}$	7.40%	1.000	0.251
$3.2 \cdot 10^{-4}$	7.20%	0.995	0.251
$1.0 \cdot 10^{-3}$	6.80%	0.926	0.257
$3.2 \cdot 10^{-3}$	6.45%	0.759	0.253
$1.0 \cdot 10^{-2}$	6.05%	0.596	0.253
$3.2 \cdot 10^{-2}$	6.05%	0.471	0.245
$1.0 \cdot 10^{-1}$	5.85%	0.417	0.248
$3.2 \cdot 10^{-1}$	6.00%	0.393	0.261
1.0	6.05%	**0.391**	0.258

The related plot of the actual DCF and minimum DCF in terms of C is the following:



The SVM model, togheter with a quadratic polynomial kernel, shows better results for the error rate, which is decreased by 2-3 percentage points, and for both the actual DCF and the minimum DCF, which maintain the trend of the previous case, but with lower values, especially for the minimum DCF, whose better value is 0.245 with $C = 3.2 \cdot 10^{-2}$.
As for the linear case, the level of miscalibration is high but it's significantly reduced (even if it's still non negligible) for high values of C.

If we compare this model to the other models that provide a quadratic separation rule, that is, the quadratic logistic regression and the MVG model, we can see that the results are quite similar (both in terms of error rate and DCF) to the ones of the quadratic logistic regression. In particular, the minimum DCF is almost the same, but logistic regression is able to better contain the miscalibration, having smaller values of the actual DCF for low values of λ. The same goes for the MVG model, which is again coherent with SVM, having a similar error rate, a slighly higher minimum DCF but better actual DCF.

6.3 SVM with Radial Basis Function (RBF) kernel

The Radial Basis Function kernel is another kernel function for SVM that allows performing potentially infinite-dimansional classifications. It is characterized by the equation

$$k(x_1, x_2) = e^{-\gamma ||x_1 - x_2||^2}$$

where γ is an hyperparameter that, similarly to C, acts as a trade-off between the model ability to generalize the data and its acceptance of misclassifications.
We try to apply this model by setting different values of C and γ in order to find a suitable combination of these two parameters.
The related results are in the following table:

γ	C	Error rate	actDCF	minDCF
e^{-4}	$1.0 \cdot 10^{-3}$	10.25%	1.000	0.357
e^{-4}	$3.2 \cdot 10^{-3}$	9.95%	1.000	0.357
e^{-4}	$1.0 \cdot 10^{-2}$	8.85%	1.000	0.50
e^{-4}	$3.2 \cdot 10^{-2}$	8.60%	0.992	0.331
e^{-4}	$1.0 \cdot 10^{-1}$	8.15%	0.890	0.331
e^{-4}	$3.2 \cdot 10^{-1}$	7.90%	0.726	0.328
e^{-4}	1.0	7.15%	0.633	0.296

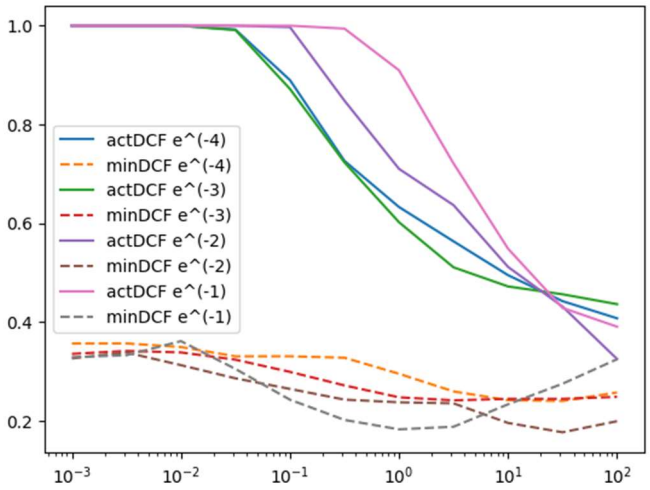
e^{-4}	3.2	6.50%	0.564	0.260
e^{-4}	$1.0 \cdot 10^{1}$	6.15%	0.495	0.242
e^{-4}	$3.2 \cdot 10^{1}$	5.60%	0.443	0.241
e^{-4}	$1.0 \cdot 10^{2}$	5.80%	0.408	0.258
e^{-3}	$1.0 \cdot 10^{-3}$	10.05%	1.000	0.336
e^{-3}	$3.2 \cdot 10^{-3}$	8.50%	1.000	0.342
e^{-3}	$1.0 \cdot 10^{-2}$	7.95%	1.000	0.339
e^{-3}	$3.2 \cdot 10^{-2}$	7.80%	0.991	0.324
e^{-3}	$1.0 \cdot 10^{-1}$	7.25%	0.872	0.299
e^{-3}	$3.2 \cdot 10^{-1}$	6.90%	0.723	0.272
e^{-3}	1.0	6.35%	0.602	0.248
e^{-3}	3.2	5.65%	0.511	0.242
e^{-3}	$1.0 \cdot 10^{1}$	5.50%	0.472	0.245
e^{-3}	$3.2 \cdot 10^{1}$	5.55%	0.456	0.245
e^{-3}	$1.0 \cdot 10^{2}$	5.30%	0.437	0.249
e^{-2}	$1.0 \cdot 10^{-3}$	8.95%	1.000	0.327
e^{-2}	$3.2 \cdot 10^{-3}$	7.40%	1.000	0.339
e^{-2}	$1.0 \cdot 10^{-2}$	7.25%	1.000	0.314
e^{-2}	$3.2 \cdot 10^{-2}$	6.85%	1.000	0.286
e^{-2}	$1.0 \cdot 10^{-1}$	6.70%	0.997	0.265
e^{-2}	$3.2 \cdot 10^{-1}$	5.85%	0.848	0.243
e^{-2}	1.0	5.40%	0.710	0.237
e^{-2}	3.2	5.00%	0.637	0.236
e^{-2}	$1.0 \cdot 10^{1}$	4.80%	0.512	0.196
e^{-2}	$3.2 \cdot 10^{1}$	4.45%	0.431	**0.177**
e^{-2}	$1.0 \cdot 10^{2}$	4.15%	0.325	0.200
e^{-1}	$1.0 \cdot 10^{-3}$	6.90%	1.000	0.329
e^{-1}	$3.2 \cdot 10^{-3}$	6.80%	1.000	0.333
e^{-1}	$1.0 \cdot 10^{-2}$	6.55%	1.000	0.362
e^{-1}	$3.2 \cdot 10^{-2}$	6.20%	1.000	0.305
e^{-1}	$1.0 \cdot 10^{-1}$	5.25%	1.000	0.243
e^{-1}	$3.2 \cdot 10^{-1}$	4.65%	0.994	0.202
e^{-1}	1.0	4.30%	0.910	0.183
e^{-1}	3.2	4.30%	0.721	0.189
e^{-1}	$1.0 \cdot 10^{1}$	**4.10%**	0.549	0.233
e^{-1}	$3.2 \cdot 10^{1}$	4.45%	0.429	0.276
e^{-1}	$1.0 \cdot 10^{2}$	5.00%	**0.390**	0.325

The previous table shows that we can achieve overall good results for or application by using the RBF kernel combining both higher values of γ and C.
In particular:

- The best error rate achieved by this model (and also the best one so far) is 4.10%, corresponding to the values $\gamma = e^{-1}$ and $C = 10$.
- The best actual DCF achieved by this model is 0.390, corresponding to the values $\gamma = e^{-1}$ and $C = 100$.

- The best minimum DCF achieved by this model (and also the best one so far) is 0.177, corresponding to the values $\gamma = e^{-2}$ and $C = 3.2 \cdot 10^{1}$.

These results related to the actual DCF and minimum DCF can be better understood if plotted as functions of C for different values of γ.



The plot shows that higher values of γ correspond to actual DCF that produce worse performances for low values of C but better performances for high values of C (and vice versa). As always and for all values of γ, setting the model with high values of C reduces significantly the level of miscalibration, which on the contrary is much higher for lower values of C, for which the classification has the lowest possible performance, corresponding to actDCF $= 1.000$.

7. Gaussian Mixture Models (GMM)

Gaussian Mixture Models (GMM) allow to perform density estimation tasks by approximating a generic non-gaussian probability distribution $f_X(x)$ as a weighed sum of gaussian distributions in the form

$$f_X(x) = \sum_{c=1}^{K} w_c \, N(x; \mu_c, \Sigma_c)$$

In particular, $f_X(x)$ represents the distribution of the samples of a dataset $D = \{x_1, \dots, x_n\}$, that the model assumes to be divided into K clusters, each one of which is responsible for generating the samples x_i.

GMM can also be used for classification purposes by applying it to approximate each class likelihood distibution $f_{X|C}(x|c)$ and then using it to calculate the related posterior probability using the Bayes theorem like it's done for the MVG model.
In particular, we can consider three different variations of the model:

- **Full covariance GMM**: default model with no assumptions on the covariance matrices;
- **Diagonal GMM**: we assume that the covariance matrices of each cluster of a same class are all diagonal;
- **Tied GMM**: we assume that the clusters of a same class have the same covariance matrix.

We now apply the full covariance GMM for classification purposes on our dataset, using a different number of components (clusters) for each class. The following table contains the obtained results, specifying for each cell, in order, the error rate, the actual DCF and the minimum DCF.

	1	**2**	**4**	**8**	**16**	**32**
1	13.75% 0.305 0.263	13.75% 0.305 0.265	9.90% 0.237 0.214	9.05% 0.196 0.185	9.95% 0.206 **0.150**	11.05% 0.227 0.185
2	10.05% 0.232 0.218	10.15% 0.234 0.217	7.70% 0.226 0.223	7.90% 0.213 0.186	8.35% 0.198 0.170	9.40% 0.227 0.186
4	10.35% 0.246 0.233	10.25% 0.236 0.232	8.00% 0.240 0.216	7.55% 0.206 0.189	7.80% 0.187 0.174	9.55% 0.238 0.187
8	7.25% 0.200 0.176	7.20% 0.191 0.181	**5.95%** 0.199 0.196	6.05% 0.193 0.179	6.25% **0.172** 0.153	7.15% 0.190 0.175
16	6.95% 0.178 0.167	6.80% 0.175 0.166	6.00% 0.192 0.192	5.95% 0.205 0.175	6.05% 0.177 0.163	7.00% 0.195 0.175
32	7.30% 0.258 0.257	7.40% 0.260 0.256	6.50% 0.282 0.246	6.05% 0.225 0.219	6.10% 0.218 0.194	7.30% 0.250 0.234

The table shows that the full covariance GMM works quite efficiently on our evaluation set. In particular, we can observe that the values of the actual DCF and minimum DCF are not only low, but

they are also very close to each other, showing that this model, for this application, has an high level of calibration.
For the first time we also have a model for which the actual DCF corresponds exactly to the minimum DCF; this happens with 16 components for the distribution $f_{X|C}(x|h_0)$ and 4 components for the distribution $f_{X|C}(x|h_1)$, where actDCF = minDCF = 0.192.
Anyway, the best result in terms of actual DCF is with 8 components for class h_0 and 16 components for class h_1 (actDCF = 0.172), and the best result in terms of minimum DCF is with 1 component for class h_0 and 16 components for class h_1 (minDCF = 0.150). These results are the best ones obtained so far considering also all the previous models (both the generative and the discriminative ones).

We now apply the diagonal GMM, from which we obtain the following results:

	1	**2**	**4**	**8**	**16**	**32**
1	13.60% 0.302 0.257	13.75% 0.305 0.261	9.75% 0.226 0.210	9.90% 0.221 0.206	8.70% 0.181 0.143	9.10% 0.197 0.137
2	11.65% 0.272 0.245	11.85% 0.267 0.249	8.95% 0.209 0.199	8.90% 0.209 0.204	7.50% 0.173 0.154	7.90% 0.181 0.144
4	6.75% 0.150 0.145	6.90% 0.161 0.154	5.65% 0.169 0.148	5.60% 0.152 0.140	5.65% 0.169 0.137	5.60% 0.168 0.141
8	6.85% 0.176 0.173	7.00% 0.187 0.176	5.60% 0.183 0.158	5.45% 0.180 0.146	**5.05%** **0.149** 0.132	5.20% 0.152 **0.131**
16	7.10% 0.222 0.207	7.10% 0.223 0.204	5.85% 0.205 0.196	5.90% 0.214 0.198	5.25% 0.177 0.162	5.40% 0.180 0.164
32	7.05% 0.196 0.175	7.10% 0.197 0.174	5.90% 0.198 0.193	5.95% 0.199 0.193	5.95% 0.207 0.185	5.95% 0.199 0.177

The diagonal GMM shows even better results than the full covariance GMM. Also in this case the model has an high level of calibration for the given application and the error rate is generally sligthly lower. This time the best actual DCF is with 8 components for class h_0 and 16 components for class h_1 (actDCF = 0.149), and the best minimum DCF is with 8 components for class h_0 and 32 components for class h_1 (minDCF = 0.131). These values are both better than the respective ones obtained for the full covariance GMM.

Since the diagonal GMM assumes uncorrelation among the features of the samples of a same cluster and of a same class, the obtained results are coherent with the dataset characteristics, being that, when we applied the naive gaussian model, we already proved the low correlation among the samples of a same class and, consequently, it's also possible (but not always true) that there is also low correlation among a subset (cluster) of the samples of a same class (which is the case of the diagonal GMM). This would favor better results for our application in terms of DCF when applying the diangonal GMM, like it was for the naive gaussian model, which, in any case, has a similar assumption (but not exacly the same).

8. Classification models recap

8.1 Best performing model for our application

So far we have employed several classification models on our dataset and for a given application ($\tilde{\pi} = 0.1$). These models were both generative (MVG, tied gaussian, naive gaussian and GMM) and discriminative (logistic regression and SVM) and provided different results in terms of error rate and DCF. We are now able to make a general sum up of the best obtained results for each model, which can be found in the following table:

Model	**Error rate**	**Best minDCF**	**Model parameters**
MVG	–	0.263	No PCA
Tied gaussian	–	0.363	No PCA
Naive gaussian	–	0.259	No PCA
Linear non prior-weighted LR	9.25%	0.361	$\lambda = 1.0 \cdot 10^{-2}$
Linear prior-weighted LR	50.40%	0.362	$\lambda = 3.2 \cdot 10^{1}$
Quadratic non prior-weighted LR	5.90%	0.244	$\lambda = 3.2 \cdot 10^{-2}$
Linear SVM	9.05%	0.358	$C = 1.0$
Polynomial SVM	7.80%	0.245	$C = 3.2 \cdot 10^{-5}$
RBF SVM	**4.45%**	0.177	$\gamma = e^{-2}$, $C = 3.2 \cdot 10^{1}$
Full covariance GMM	9.95%	0.150	$N_{h_0} = 1$, $N_{h_1} = 16$
Diagonal GMM	5.20%	**0.131**	$N_{h_0} = 8$, $N_{h_1} = 32$

This table shows that, in terms of minimum DCF, the most promising model for our application is the diagonal GMM, for which minDCF = 0.131. The related actual DCF has value 0.152 and, even if it's not the best one of the diagonal GMM (which is 0.149), it's the best actual DCF among the ones of the other models, being also close to the related minimum DCF.
The gaussian models, a part from the provided results, are characterized by a homogeneous good calibration. On the contrary, both the logistic regression and the SVM models have a calibration level with an high variability (most of which is characterized by a low calibration, especially in relation to the actual DCF), depending on the employed parameters.

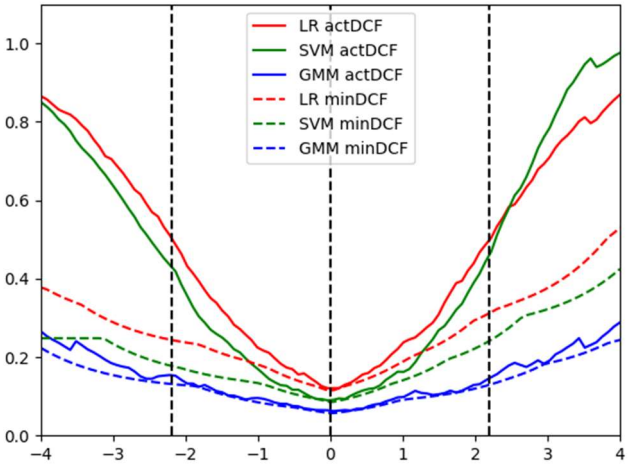
8.2 Models performance on different applications

We can also evaluate the classification models by eploying them with respect to different applications, that is, by moving the value of the application prior $\tilde{\pi}$. For this evaluation we only consider the values of actual DCF and minimum DCF provided by the following models:

- quadratic non-weighted logistic regression with $\lambda = 3.2 \cdot 10^{-2}$;
- SVM with RBF kernel and with $\gamma = e^{-2}$, $C = 3.2 \cdot 10^{1}$;
- diagonal GMM with $N_{h_0} = 8$ and $N_{h_1} = 32$;

These are the three best performing model configurations related to logistic regression, SVM and GMM as showed in the previous table.
We can plot the results of this models as a set of functions of the actualDCF and minimum DCF, where the x axis represents the value of the value of the effective prior expressed in form of a threshold with opposite sign $log \frac{\tilde{\pi}}{1-\tilde{\pi}}$.

The resulting plot is the following:



The values related to our main application are the ones corresponding to $x = log\frac{\tilde{\pi}}{1-\tilde{\pi}}$ where $\tilde{\pi} = 0.1$, that is, $x \cong -2.20$. The values at $x = 0$ are the ones corresponding to the application with $\tilde{\pi} = 0.5$ and the values at $x \cong 2.20$ are the ones corresponding to the application with $\tilde{\pi} = 0.9$.
More in general, the plot shows that the model ranking that we observed for the application (0.1, 1, 1) remains almost the same for all the other possible values of $\tilde{\pi}$ and both in terms of actual DCF and minimum DCF; the only exception is for high values of $\tilde{\pi}$, for which the SVM becomes slightly more advantageous in terms of actual DCF.
Moreover, we can observe that the best performances for all the three models correspond to the application with uniform prior ($\tilde{\pi} = 0.5$), where the results are very close and show an almost perfect level of calibration. For the other possible applications the GMM show a level of calibration that gets a little worse by getting far from $\tilde{\pi} = 0.5$ but that is still quite good. On the other hand, the other two models clearly have a great worsening in the calibration level for very high or very low values of $\tilde{\pi}$.

9. Calibration and fusion

In this last chapter we analyze the effects of calibration and fusion on our classification models. In particular, we focus on the three best models among the ones employed so far, that is:

- quadratic non-weighted logistic regression with $\lambda = 3.2 \cdot 10^{-2}$;
- SVM with RBF kernel and with $\gamma = e^{-2}$, $C = 3.2 \cdot 10^{1}$;
- diagonal GMM with $N_{h_0} = 8$ and $N_{h_1} = 32$;

9.1 Calibration

Firstly, we calibrate the scores using a K-fold approach according to the following steps (for each of the previous models):

- we choose a classification model and we employ it over the validation set, producing a set of scores;
- we divide this set of scores into K equal partitions, using $K-1$ of them for training the calibration model and the remaining one for evaluating it, producing a set of calibrated scores;
- we repeat the previous step K times using each time a different partition for evaluating the calibration model;
- we pool the K previously obtained set of calibrated scores into a single set;
- we perform again the classification using the calibrated scores, calculating the related actual DCF to compare it with the actual and minimum DCF obtained with the non-calibrated scores.

The calibration model consists of a linear logistic regression trained over the $K-1$ set of non-calibrated scores. This allows to turn a non-calibrated score s into a calibrated score $f(s) = \alpha s + \gamma$ such that

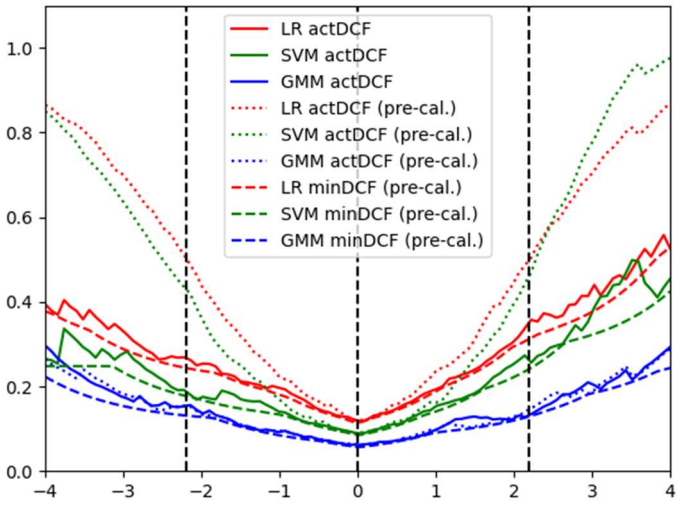
$$\alpha s + \gamma = \alpha s + \beta - log\frac{\tilde{\pi}}{1-\tilde{\pi}} = log\frac{P(C=h_1|s)}{P(C=h_0|s)} - log\frac{\tilde{\pi}}{1-\tilde{\pi}}$$

where $\tilde{\pi} = 0.1$ is referred to our main application.
By applying the score calibration on our validation set, once for each of the three chosen classification models, we obtain the following results:

	Quadratic LR		**SVM with RBF kernel**		**Diagonal GMM**	
Scores	actDCF	minDCF	actDCF	minDCF	actDCF	minDCF
Raw	0.500	0.244	0.431	0.177	0.152	0.131
Calibrated	0.272	0.248	0.188	0.179	0.150	0.133

The table shows that, for LR and SVM, the score calibration is able to produce a significant improvement in the actual DCF, which is almost halved for LR (from 0.500 to 0.272) and more than halved for SVM (from 0.431 to 0.188). The minimum DCF, on the other hand, is slightly increased in both cases, making the related actual DCF even closer.
On the contrary, the score calibration does not produce a strong impact on GMM, which was already quite well calibrated. In this case the actual DCF decrease from 0.152 to 0.150, with a slight increase in the minimum DCF (from 0.131 to 0.133), making the calibration effects negligible.

We now apply the calibrated scores over different final applications by varying the effective prior $\tilde{\pi}$. Then we plot the related values of the DCFs as a function of the effective priors expressed in form of the factor $\frac{\tilde{\pi}}{1-\tilde{\pi}}$, comparing them to the ones obtained without the score calibration.
The resulting plot is the following:



The plot shows an improvement in the calibrated actual DCF for a wide range of application priors. In particular, the calibration performed with respect to the scores of the logistic regression and SVM models produces actual DCFs which are much closer to the related values of the minimum DCF calculated for the non-calibrated scores. Again, the GMM was already quite well calibrated and, because of this, the difference between the actual DCF calculated with the calibrated scores and the one calculated with the non-calibrated scores is minimal and negligible.

Overall, we can say that, independently from the application effective prior $\tilde{\pi}$, the calibration is positive for the first two classification models and useless for the GMM.

9.2 Fusion

Finally, we consider weighting the contribution of all the three classification models employed for calibration in oder to produce a set of fused scores s_{fused} such that

$$s_{\text{fused}} = \alpha_1 s_1 + \alpha_2 s_2 + \alpha_3 s_3$$

where s_1, s_2, s_3 are the scores obtained for the validation set with LR, SVM and GMM and $\alpha_1, \alpha_2, \alpha_3$ are the weights associated with each score. The weights (and the bias term) are again estimated using a logistic regression model such that

$$s_{\text{fused}} = \alpha^T s + \gamma$$

where $\gamma = \beta - \frac{\tilde{\pi}}{1-\tilde{\pi}}$ and s is the stack of the scores obtained with the three models.
By applying the fusing method with respect to our validation set we obtain the following results:

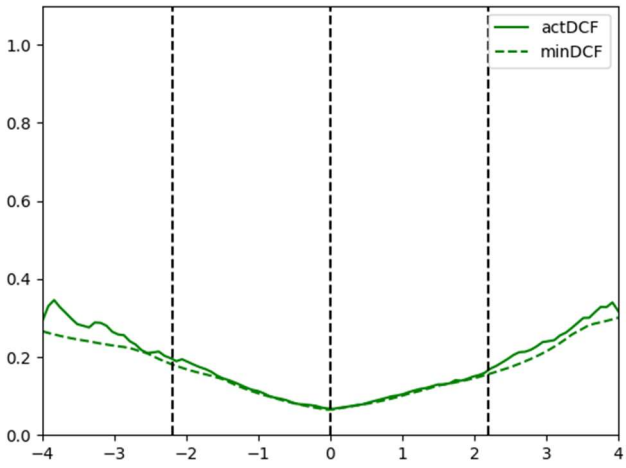
Score fusion (LR – SVM - GMM)		
Error rate	actDCF	minDCF
5.10%	0.166	0.127

10. Final evaluation

In this final chapter we employ one of the models obtained so far over an evaluation set, in order to test its goodness. The model we choose to employ is the diagonal GMM with $N_{h_0} = 8$ and $N_{h_1} = 16$. The choice of this model depend on the fact that the calibration of the diagonal GMM model with $N_{h_0} = 8$ and $N_{h_1} = 32$, which has the lowest minimum DCF of all the tested models, was not able to reduce the actual DCF enouch to be lower than the one of the chosen model, which is actDCF = 0.149 and which is the lowest actual DCF.
By testing this model on the evaluation set and for the target application ($\tilde{\pi} = 0.1$) we obtain the following results:

Diagonal GMM with $N_{h_0} = 8$ and $N_{h_1} = 32$		
Error rate	actDCF	minDCF
5.40%	0.193	0.181

The related Bayes plot considering the performance the model on different applications is the following:



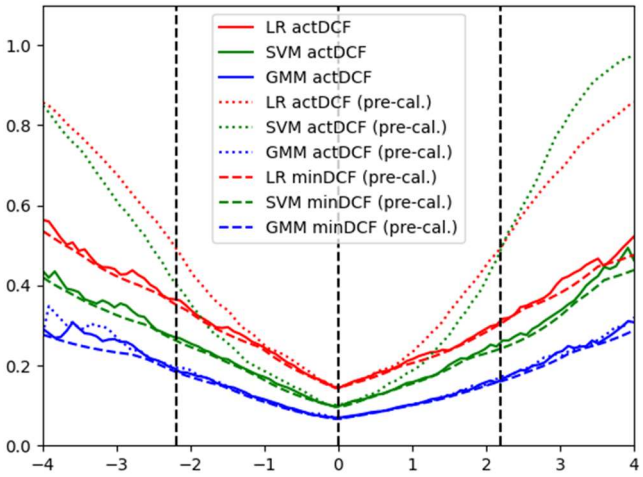
The model shows to have an overall good calibration, especially for $0.1 \leq \tilde{\pi} \leq 0.9$, for which actual DCF and minimum DCF are almost overlapped.
The results related to our target application show values of actual DCF and minimum DCF that are satisfying even though they are slightly worst than the related values obtained on the validation set. Despite this, these values are still better than the ones obtained on the validation set for all the other models, excluding diagonal GMM itself.

We now try to employ on the evaluation set the three models calibrated in the previous chapter, in order to make a comparison with the results obtained with the chosen final model.
The results are the following:

	Quadratic LR		**SVM with RBF kernel**		**Diagonal GMM**	
Scores	actDCF	minDCF	actDCF	minDCF	actDCF	minDCF
Calibrated	0.366	0.352	0.270	0.264	0.190	0.187

The table shows that the calibrated diagonal GMM with $N_{h_0} = 8$ and $N_{h_1} = 32$ performs slightly better than the chosen GMM model, having an actual DCF smaller by 0.003. The other two models (quadratic LR and SVM with RBF kernel), even when calibrated, cannot compete with GMM.

Once again, we also plot these models in order to understand their performance for different target applications. The plot is the following:

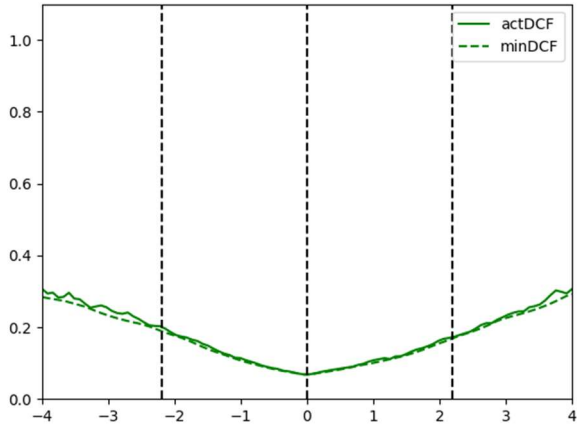


The plot gives us a confirmation on the better efficacy of the GMM with respect to LR and SVM for all the applications analyzed.

Lastly, we try to employ the same three models but this time in their fused version, in order to see if this choice gives any improvement to the classification.
In this case, the obtained results are the following:

Score fusion (LR – SVM - GMM)		
Error rate	actDCF	minDCF
6.15%	0.200	0.189

The table shows that the score fusion provides an actual DCF and a minimum DCF that are very close (even if slightly worse) to the ones provided by the calibrated diagonal GMM.
The related Bayes plot is the following:



The fused model looks quite similar to the GMM one, having a better overall calibration but also slightly worse values of the actual and minimum DCFs.